Простой ИИ врагов

Простой ИИ врагов

Проблема

У нас есть враг. Он стоит на месте. Игрок проходит мимо - враг стоит. Игрок бьёт его - враг стоит. Игрок уходит - враг стоит. Не очень впечатляет.

Нам нужно, чтобы враг вёл себя хотя бы минимально осмысленно: ходил по маршруту, замечал игрока, преследовал его, атаковал. Не нужен настоящий интеллект - достаточно создать **иллюзию** разумного поведения. Игрок не отличит простой автомат от сложной нейросети, если враг вовремя реагирует и не делает очевидно глупых вещей.

Как раз для этого мы используем конечные автоматы, поиск пути и коллизии - всё, что уже разбирали.

Патрулирование

Самое базовое поведение: враг ходит по маршруту. Туда-сюда, по кругу, между точками - неважно. Главное - он не стоит столбом.

Простейший вариант: задаём список **точек патрулирования** (waypoints). Враг идёт к первой точке, дошёл - идёт ко второй, и так далее. Добрался до последней - начинает сначала.

Python C#

```python
class PatrollingEnemy:
    def __init__(self, x, y, waypoints, speed):
        self.x = x
        self.y = y
        self.waypoints = waypoints  # list of (x, y) tuples
        self.current_wp = 0
        self.speed = speed

    def update(self, delta_time):
        target_x, target_y = self.waypoints[self.current_wp]
        dx = target_x - self.x
        dy = target_y - self.y
        dist = (dx ** 2 + dy ** 2) ** 0.5

        if dist < 2:  # close enough, move to next waypoint
            self.current_wp = (self.current_wp + 1) % len(self.waypoints)
```

```
        else:
            # normalize direction and move
            self.x += (dx / dist) * self.speed * delta_time
            self.y += (dy / dist) * self.speed * delta_time


# usage
guard = PatrollingEnemy(100, 100, waypoints=[(100, 100), (300, 100), (300,
```

Враг движется по квадрату 200x200 пикселей. `dist < 2` - порог "прибытия": если до точки осталось меньше 2 пикселей, считаем, что дошли. Без этого порога враг может бесконечно колебаться вокруг точки, не попадая в неё точно.

Точки патрулирования можно задавать вручную на уровне (в редакторе или в коде) или генерировать - например, случайные точки внутри комнаты. Для процедурно генерируемых уровней второй вариант удобнее.

Обнаружение игрока

Ну, патрулировать враг умеет. Теперь было бы хорошо, если б он замечал игрока. Самый простой способ - **радиус обнаружения**: если расстояние между врагом и игроком меньше определённого зн

```
dist = distance(enemy, player)
if dist < detection_radius:
    # enemy sees the player
```

Это, по сути, проверка столкновения двух окружностей: хитбокс игрока и невидимая "зона видимости" врага. Знакомая тема.

Python C#

```
def distance(ax, ay, bx, by):
    return ((ax - bx) ** 2 + (ay - by) ** 2) ** 0.5

def can_detect(enemy, player, radius):
    return distance(enemy.x, enemy.y, player.x, player.y) < radius


# usage
if can_detect(guard, player, radius=200):

```

```
    # switch to chase
    pass
```

Это работает, но есть проблема: враг видит сквозь стены. Игрок за толстой стеной, в соседней комнате, но попадает в радиус - и враг его "видит". Это выглядит нечестно.

Проверка видимости (Line of Sight)

Чтобы враг не видел сквозь стены, нужно проверить, есть ли между врагом и игроком препятствие. Простой способ: пройтись от врага к игроку маленькими шагами и проверить, не попали ли мы в стену. Это, по сути, **рейкаст** (raycast) - "бросаем луч" от врага к игроку.

Если мы работаем с тайловой картой, то проверяем, не проходит ли луч через тайл-стену:

Python C#

```python
def has_line_of_sight(x1, y1, x2, y2, level, tile_size):
    dx = x2 - x1
    dy = y2 - y1
    dist = (dx ** 2 + dy ** 2) ** 0.5
    if dist == 0:
        return True

    steps = int(dist / (tile_size / 2))  # check every half-tile
    for i in range(1, steps):
        t = i / steps
        check_x = x1 + dx * t
        check_y = y1 + dy * t
        col = int(check_x // tile_size)
        row = int(check_y // tile_size)
        if level[row][col] == 1:  # wall
            return False
    return True


# combined check: in range AND no walls between
def can_see_player(enemy, player, radius, level, tile_size):
    if distance(enemy.x, enemy.y, player.x, player.y) > radius:
        return False
    return has_line_of_sight(enemy.x, enemy.y, player.x, player.y, level, t
```

Мы проходим от врага к игроку шагами в пол-тайла и проверяем каждую точку. Если хоть одна попала в стену - враг не видит игрока. Шаг в пол-тайла - компромисс: слишком крупный шаг может

"перешагнуть" тонкую стену, слишком мелкий - медленно.

Это упрощённый рейкаст. Существуют более точные алгоритмы (алгоритм Брезенхэма, DDA и другие). В этом простом варианте, к примеру, враг может "видеть" сквозь угол стены.

Преследование

Враг заметил игрока. Что дальше? Бежать к нему. Наивный подход - двигаться напрямую к координатам игрока:

```
direction = normalize(player.pos - enemy.pos)
enemy.pos += direction * speed * delta_time
```

Это работает на открытом пространстве. Но если между врагом и игроком стена, враг упрётся в неё и будет стоять, дёргаясь. Не очень умно выглядит.

Решение: использовать поиск пути. Строим маршрут от врага к игроку (например, через A* или алгоритм Дейкстры на тайловой сетке) и двигаемся по найденному пути.

Python C#

```python
class ChasingEnemy:
    def __init__(self, x, y, speed):
        self.x = x
        self.y = y
        self.speed = speed
        self.path = []  # list of (x, y) waypoints from pathfinder

    def chase(self, player, level, tile_size):
        # rebuild path periodically
        start = (int(self.x // tile_size), int(self.y // tile_size))
        goal = (int(player.x // tile_size), int(player.y // tile_size))
        self.path = find_path(level, start, goal)  # your A* / BFS

    def update(self, delta_time):
        if not self.path:
            return

        target_x, target_y = self.path[0]
        dx = target_x - self.x
        dy = target_y - self.y
        dist = (dx ** 2 + dy ** 2) ** 0.5

        if dist < 2:
            self.path.pop(0)
```

```
        else:
            self.x += (dx / dist) * self.speed * delta_time
            self.y += (dy / dist) * self.speed * delta_time
```

Но пересчитывать путь **каждый кадр** - дорого. Игрок двигается, но не настолько быстро, чтобы пересчитывать маршрут 60 раз в секунду. Обычно пересчитывают раз в 0.5-1 секунду или когда игрок переместился на определённое расстояние от последней точки.

Атака

Враг добрался до игрока. Теперь нужно атаковать. Два вопроса: когда атаковать и как не атаковать слишком часто.

Когда: если расстояние до игрока меньше **радиуса атаки** (значительно меньше радиуса обнаружения).

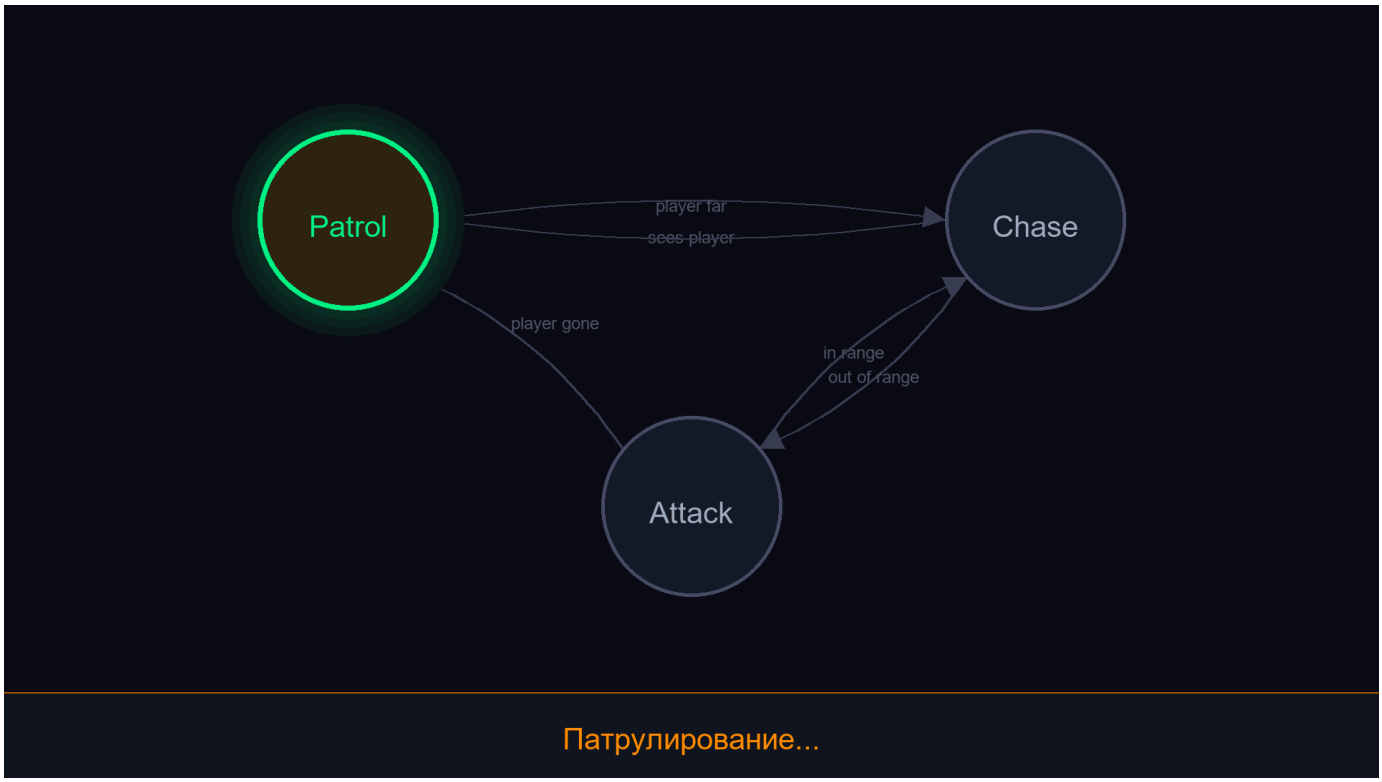
Как не слишком часто: вводим **кулдаун** (cooldown) - минимальное время между атаками.

Если вы реализовали событийную модель, атака - хороший кандидат на событие: `on_attack.fire(player, damage)` . Тогда звуковая система проиграет звук, система анимаций запустит анимацию удара, UI обновит полоску здоровья - и всё это без того, чтобы AttackBehavior знал обо всех этих системах.

Собираем всё: автомат врага

Теперь у нас есть все кусочки: патрулирование, обнаружение, преследование, атака. Как их связать? Правильно - конечный автомат. Состояния врага:

- **Patrol** - ходит по маршруту. Если видит игрока -> Chase.
- **Chase** - бежит к игроку. Если игрок близко -> Attack. Если игрок далеко (убежал) -> Patrol.
- **Attack** - бьёт игрока. Если игрок отошёл -> Chase. Если игрок далеко -> Patrol.



Можно реализовать это через классы-состояния (как в разделе про автоматы).

Обратите внимание на несколько деталей:

- `lose_radius > detect_radius` . Враг замечает игрока на расстоянии 200, но прекращает погоню на 350. Без этого враг на границе радиуса будет бесконечно переключаться между Patrol и Chase - увидел, потерял, увидел, потерял. Этот зазор называют **гистерезис**.
- **Скорость погони выше** (`speed * 1.5`). Враг при преследовании ускоряется. Иначе, если его скорость равна скорости игрока, он никогда не догонит - и это выглядит абсурдно.
- `attack_range * 1.5` для выхода из Attack. Тот же гистерезис: чтобы враг не дёргался между Attack и Chase, когда игрок стоит на границе дистанции атаки.

Гистерезис - главный секрет того, чтобы ИИ выглядел "спокойным" и уверенным. Без него враги постоянно дёргаются, переключаясь между состояниями на каждом кадре. Используйте разные пороги для входа в состояние и для выхода из него.

Настройка поведения

Меняя параметры, из одного и того же автомата можно получить совершенно разных врагов:

Параметр	Стражник	Берсерк	Лучник
Скорость	60	120	50
Радиус обнаружения	150	300	250

Параметр	Стражник	Берсерк	Лучник
Радиус потери	250	500	400
Дистанция атаки	40	50	200
Кулдаун атаки	1.5 сек	0.5 сек	2.0 сек
Урон	10	25	15

Хм. Стражник - медленный, видит недалеко, бьёт средне. Берсерк - быстрый, видит далеко, бьёт часто и больно. Лучник - медленный, но стреляет издалека. Та же логика, разные числа.

Это хороший пример того, зачем нужна архитектура: когда поведение отделено от данных, создание новых типов врагов - это не написание нового кода, а подстановка других чисел. В разумных пределах.

Типичные ошибки

Несколько вещей, которые часто идут не так (+ к типичными ошибками):

- **Враг видит сквозь стены.** Забыли проверку видимости (line of sight). Используйте рейкаст.
- **Враг дёргается на границе состояний.** Нет гистерезиса. Используйте разные пороги для входа и выхода.
- **Все враги пересчитывают путь каждый кадр.** 50 врагов x 60 FPS = 3000 вызовов A* в секунду. Пересчитывайте раз в 0.5-1 сек или по необходимости.
- **Скорость врага не зависит от deltaTime.** На быстром компьютере враг бегает в два раза быстрее. Умножайте на delta_time.
- **Враг продолжает преследовать за пределами уровня.** Ограничивайте координаты врага так же, как ограничиваете камеру.

Что дальше?

Описанный автомат - базовый, но расширяемый. Несколько направлений для усложнения:

- **Убегание.** Состояние Flee: если здоровье ниже порога, враг бежит от игрока. Направление - от игрока, всё остальное как в Chase, но наоборот.
- **Группы.** Один враг замечает игрока и "зовёт" соседей через событие: `bus.fire("alert", player_position)` . Остальные враги подписаны и переключаются в Chase.
- **Разные виды обнаружения.** Конус зрения вместо круга. Звук шагов (больший радиус обнаружения, но не требует line of sight).
- **Задержка реакции.** Враг не мгновенно переключается в Chase - сначала "удивляется" (помним, как в MGS: 0.5 сек пауза + восклицательный знак над головой). Простой таймер, но создаёт ощущение,

что враг живой.

- **Управляющие силы (Steering).** Вместо мгновенного разворота - плавное накопление сил: seek, flee, arrive, wander. А поверх этого - Boids: три простых правила создают стаю. Подробнее: Управляющие силы и Boids.
- **Автоматический подбор параметров.** Таблицу Стражник/Берсерк/Лучник мы заполнили вручную. А что, если типов врагов 50? Генетический алгоритм может найти сбалансированные параметры сам - через эволюцию популяции кандидатов.
- **Нейросетевой ИИ.** Вместо правил "если HP < 30% - лечиться" - нейросеть, которая учится принимать решения из опыта. Подаём на вход (HP, расстояние, состояние), получаем действие.
- **Вероятностные переходы.** FSM предсказуем: если видит - всегда Chase. Цепь Маркова добавляет случайность: 70% Chase, 20% Attack, 10% Flee - враг становится менее "читаемым".

Ок. Каждое из этих улучшений - ещё одно состояние или параметр в уже существующем автомате. Конечные автоматы хороши тем, что новое поведение - это новая стрелочка на диаграмме, а не переписывание всего кода.

А если состояний становится слишком много и переходы превращаются в паутину - стоит посмотреть на деревья поведений (Behavior Trees). Это следующий шаг: вместо графа переходов - дерево приоритетов, где добавить поведение = добавить ветку.

Powered by Wiki.js